

Flutter

Basic Widgets — Bangla Detailed Guide

Flutter-এর সকল বেসিক Widget বাংলায় বিস্তারিত আলোচনা

Md Aslam Howlader | ID: 20245103012 | BUBT
Course: Computer Networks & Mobile App Development

Flutter Widget কী?

Flutter-এ সবকিছুই Widget। একটি Flutter অ্যাপ মূলত Widget-এর একটি বড় গাছ (Widget Tree)। Button, Text, Image, Layout — সবকিছুই Widget দিয়ে তৈরি।

Widget দুই ধরনের:

- StatelessWidget — যার নিজস্ব কোনো পরিবর্তনযোগ্য অবস্থা (State) নেই। একবার তৈরি হলে আর পরিবর্তন হয় না।
- StatefulWidget — যার State আছে এবং setState() ব্যবহার করে UI পরিবর্তন করা যায়।

এই ডকুমেন্টে Flutter-এর সবচেয়ে গুরুত্বপূর্ণ Basic Widget গুলো বাংলায় বিস্তারিত ব্যাখ্যা করা হয়েছে, প্রতিটির সাথে Code Example সহ।

১. Text Widget

Text Widget হলো Flutter-এর সবচেয়ে মৌলিক Widget। এটি স্ক্রিনে যেকোনো লেখা প্রদর্শন করতে ব্যবহৃত হয়।

সংজ্ঞা (Definition)

Text Widget একটি single line বা multiple line লেখা রেন্ডার করে। এটি style, alignment, overflow সহ অনেক property সাপোর্ট করে।

গুরুত্বপূর্ণ Properties

Property	ধরন (Type)	কাজ (Purpose)
data	String	প্রদর্শনযোগ্য লেখা
style	TextStyle	ফন্ট, রঙ, সাইজ নির্ধারণ
textAlign	TextAlign	বাম, ডান, মাঝে বা justify
maxLines	int	সর্বোচ্চ কতটি লাইন দেখাবে
overflow	TextOverflow	লেখা বড় হলে কী করবে (ellipsis)
softWrap	bool	লেখা নতুন লাইনে যাবে কিনা

Code উদাহরণ

```
// সাধারণ Text
Text('আমি Flutter শিখছি')

// Styled Text
Text(
  'Md Aslam Howlader',
  style: TextStyle(
    fontSize: 24,
    fontWeight: FontWeight.bold,
    color: Colors.blue,
    letterSpacing: 1.5,
    decoration: TextDecoration.underline,
  ),
  textAlign: TextAlign.center,
  maxLines: 2,
  overflow: TextOverflow.ellipsis,
)
```

```
// RichText - একাধিক style এক লাইনে
RichText(
  text: TextSpan(children: [
    TextSpan(text: 'Hello ', style: TextStyle(color: Colors.black)),
    TextSpan(text: 'Flutter', style: TextStyle(color: Colors.blue, fontWeight:
FontWeight.bold)),
  ]),
)
```

ব্যবহার: শিরোনাম, বডি টেক্সট, লেবেল, বার্তা — সব ধরনের লেখা প্রদর্শনে Text ব্যবহার হয়।

২. Container Widget

Container হলো Flutter-এর সবচেয়ে বহুল ব্যবহৃত Widget। এটি একটি বাক্সের মতো — যার মধ্যে অন্য Widget রাখা যায় এবং বাক্সটির আকার, রঙ, border, padding, margin সব নিয়ন্ত্রণ করা যায়।

গুরুত্বপূর্ণ Properties

Property	Type	কাজ
width / height	double	Container-এর প্রস্থ ও উচ্চতা নির্ধারণ
color	Color	পটভূমির রঙ
padding	EdgeInsets	ভেতরের ফাঁকা জায়গা
margin	EdgeInsets	বাইরের ফাঁকা জায়গা
decoration	BoxDecoration	রঙ, border, borderRadius, shadow
alignment	Alignment	ভেতরের child কোথায় থাকবে
child	Widget	Container-এর ভেতরের Widget

Code উদাহরণ

```
Container(
  width: 200,
  height: 100,
  margin: EdgeInsets.all(16),
  padding: EdgeInsets.symmetric(horizontal: 20, vertical: 10),
  alignment: Alignment.center,
  decoration: BoxDecoration(
    color: Colors.blue.shade100,
    borderRadius: BorderRadius.circular(12),
    border: Border.all(color: Colors.blue, width: 2),
    boxShadow: [
      BoxShadow(color: Colors.grey, blurRadius: 8, offset: Offset(2, 4)),
    ],
  ),
  child: Text('আমি Container-এর ভেতরে আছি'),
)
```

গুরুত্বপূর্ণ: color এবং decoration একসাথে ব্যবহার করা যায় না। decoration ব্যবহার করলে color সেখানেই দিতে হবে (decoration-এর মধ্যে)।

৩. Row ও Column Widget

Row এবং Column হলো Flutter-এর প্রধান layout Widget। Row বাম থেকে ডানে সাজায় (horizontal), আর Column উপর থেকে নিচে সাজায় (vertical)।

Row Widget

Row সব child Widget গুলোকে horizontally (পাশাপাশি) সাজায়।

Column Widget

Column সব child Widget গুলোকে vertically (উপর-নিচ) সাজায়।

MainAxis ও CrossAxis

Row-এর mainAxis হলো horizontal (বাম-ডান) এবং crossAxis হলো vertical (উপর-নিচ)।

Column-এর mainAxis হলো vertical (উপর-নিচ) এবং crossAxis হলো horizontal (বাম-ডান)।

Property	সম্ভাব্য মান	কাজ
mainAxisAlignment	start, end, center, spaceBetween, spaceAround, spaceEvenly	মূল অক্ষে সাজানোর নিয়ম
crossAxisAlignment	start, end, center, stretch	আড়াআড়ি অক্ষে সাজানো
mainAxisSize	max, min	কতটুকু জায়গা নেবে

Code উদাহরণ

```
// Row উদাহরণ
Row(
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
  crossAxisAlignment: CrossAxisAlignment.center,
  children: [
    Icon(Icons.person, size: 30),
    Text('Md Aslam'),
    ElevatedButton(onPressed: () {}, child: Text('Follow')),
  ],
)
```

```
// Column উদাহরণ
Column(
  mainAxisAlignment: MainAxisAlignment.center,
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    Text('নাম: Md Aslam', style: TextStyle(fontSize: 18)),
    SizedBox(height: 8),
    Text('ID: 20245103012'),
    SizedBox(height: 8),
    Text('বিশ্ববিদ্যালয়: BUBT'),
  ],
)
```

8. Stack Widget

Stack Widget একটির উপর আরেকটি Widget স্তরে স্তরে (layer by layer) রাখতে দেয়। ছবির উপর লেখা, badge যোগ করা — এই ধরনের কাজে Stack ব্যবহার হয়।

গুরুত্বপূর্ণ Properties

- alignment — child গুলো কোথায় থাকবে (default: topLeft)
- fit — child কতটুকু জায়গা নেবে (loose বা expand)
- clipBehavior — বাইরে গেলে কাটবে কিনা

Positioned Widget

Stack-এর মধ্যে Positioned Widget ব্যবহার করে নির্দিষ্ট অবস্থানে রাখা যায়।

```
Stack(  
  alignment: Alignment.center,  
  children: [  
    // পেছনের layer — ছবি  
    Container(width: 200, height: 200, color: Colors.blue),  
  
    // মাঝের layer — আইকন  
    Icon(Icons.person, size: 80, color: Colors.white),  
  
    // সামনের layer — badge  
    Positioned(  
      top: 10,  
      right: 10,  
      child: Container(  
        padding: EdgeInsets.all(4),  
        decoration: BoxDecoration(color: Colors.red, shape: BoxShape.circle),  
        child: Text('3', style: TextStyle(color: Colors.white)),  
      ),  
    ),  
  ],  
)
```

৫. Scaffold Widget

Scaffold হলো একটি পূর্ণ পর্দার (full screen) কাঠামো। এটি Material Design-এর মূল structure দেয় — AppBar, Body, BottomBar, Drawer, FAB সব একসাথে।

গুরুত্বপূর্ণ Properties

Property	কাজ
appBar	উপরের নেভিগেশন বার — title, actions সহ
body	মূল কন্টেন্ট এলাকা
drawer	বাম দিক থেকে স্লাইড করা মেনু
floatingActionButton	স্ক্রিনের কোণে ভাসমান বাটন
bottomNavigationBar	নিচের নেভিগেশন বার
backgroundColor	পুরো স্ক্রিনের পটভূমি রঙ
bottomSheet	নিচ থেকে উঠে আসা প্যানেল

Code উদাহরণ

```
Scaffold(
  appBar: AppBar(
    title: Text('আমার অ্যাপ'),
    backgroundColor: Colors.blue,
    actions: [
      IconButton(icon: Icon(Icons.search), onPressed: () {}),
      IconButton(icon: Icon(Icons.notifications), onPressed: () {}),
    ],
  ),
  drawer: Drawer(
    child: ListView(children: [
      DrawerHeader(child: Text('মেনু'), decoration: BoxDecoration(color:
Colors.blue)),
      ListTile(leading: Icon(Icons.home), title: Text('হোম')),
      ListTile(leading: Icon(Icons.person), title: Text('প্রোফাইল')),
    ]),
  ),
  body: Center(child: Text('মূল কন্টেন্ট এখানে')),
  floatingActionButton: FloatingActionButton(
    onPressed: () {},
    child: Icon(Icons.add),
  ),
  bottomNavigationBar: BottomNavigationBar(
    items: [
```

```
BottomNavigationBarItem(icon: Icon(Icons.home), label: 'হোম'),  
BottomNavigationBarItem(icon: Icon(Icons.person), label: 'প্রোফাইল'),  
BottomNavigationBarItem(icon: Icon(Icons.settings), label: 'সেটিংস'),  
],  
,  
)
```

৬. Button Widgets

Flutter-এ বিভিন্ন ধরনের Button Widget আছে। প্রতিটির নিজস্ব visual style ও purpose আছে।

৬.১ ElevatedButton

উঁচু করা (raised) বাটন — সবচেয়ে বেশি ব্যবহৃত। Primary action-এর জন্য ব্যবহার হয়।

```
ElevatedButton(  
  onPressed: () { print('চাপা হয়েছে!'); },  
  style: ElevatedButton.styleFrom(  
    backgroundColor: Colors.blue,  
    foregroundColor: Colors.white,  
    padding: EdgeInsets.symmetric(horizontal: 30, vertical: 15),  
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(10)),  
  ),  
  child: Text('সাবমিট করুন', style: TextStyle(fontSize: 16)),  
)
```

৬.২ TextButton

সমতল (flat) বাটন — কোনো background বা border নেই। Secondary action-এর জন্য।

```
TextButton(  
  onPressed: () {},  
  child: Text('আরো দেখুন'),  
)
```

৬.৩ OutlinedButton

শুধু border আছে, background নেই — alternative action-এর জন্য।

```
OutlinedButton(  
  onPressed: () {},  
  style: OutlinedButton.styleFrom(side: BorderSide(color: Colors.blue, width: 2)),  
  child: Text('বাতিল করুন'),  
)
```

৬.৪ IconButton

শুধু আইকন দিয়ে তৈরি বাটন — AppBar-এর actions-এ বা toolbar-এ ব্যবহৃত।

```
IconButton(  
  icon: Icon(Icons.favorite, color: Colors.red),  
  iconSize: 30,
```

```
tooltip: 'পছন্দ করুন',  
onPressed: () {},  
)
```

৬.৫ FloatingActionButton

স্ক্রিনে ভাসমান বৃত্তাকার বাটন — সবচেয়ে গুরুত্বপূর্ণ কাজের জন্য।

```
FloatingActionButton(  
  onPressed: () {},  
  backgroundColor: Colors.green,  
  child: Icon(Icons.add, size: 28),  
)  
  
// Extended FAB — লেখা সহ  
FloatingActionButton.extended(  
  onPressed: () {},  
  icon: Icon(Icons.add),  
  label: Text('নতুন যোগ করুন'),  
)
```

৭. Image Widget

Image Widget স্ক্রিনে ছবি দেখাতে ব্যবহার হয়। বিভিন্ন উৎস থেকে ছবি লোড করা যায়।

ছবির উৎস (Sources)

- `Image.asset()` — অ্যাপের ভেতরে রাখা ছবি (assets folder)
- `Image.network()` — ইন্টারনেট থেকে URL দিয়ে ছবি
- `Image.file()` — ডিভাইসের Storage থেকে ছবি
- `Image.memory()` — Uint8List থেকে ছবি (bytes)

গুরুত্বপূর্ণ Properties

Property	Type	কাজ
width / height	double	ছবির আকার নির্ধারণ
fit	BoxFit	ছবি কীভাবে fit হবে
color	Color	ছবির উপর color filter
loadingBuilder	Function	লোড হওয়ার সময় placeholder
errorBuilder	Function	লোড না হলে কী দেখাবে

BoxFit মানগুলো

- `BoxFit.cover` — ছবি পুরো জায়গা ঢেকে ফেলবে (crop হতে পারে)
- `BoxFit.contain` — ছবি পুরো দেখাবে (empty space থাকতে পারে)
- `BoxFit.fill` — stretch করে পুরো জায়গা পূরণ করবে
- `BoxFit.fitWidth` — width-এর সাথে মেলাবে
- `BoxFit.fitHeight` — height-এর সাথে মেলাবে

Code উদাহরণ

```
// Asset Image (pubspec.yaml-এ declare করতে হবে)
Image.asset(
  'assets/images/profile.png',
  width: 150,
  height: 150,
  fit: BoxFit.cover,
```

```
)

// Network Image
Image.network(
  'https://example.com/photo.jpg',
  width: 200,
  fit: BoxFit.cover,
  loadingBuilder: (ctx, child, loadingProgress) {
    if (loadingProgress == null) return child;
    return CircularProgressIndicator();
  },
  errorBuilder: (ctx, error, stack) => Icon(Icons.broken_image, size: 50),
)

// CircleAvatar – গোলাকার ছবি
CircleAvatar(
  radius: 40,
  backgroundImage: NetworkImage('https://example.com/avatar.jpg'),
  backgroundColor: Colors.grey.shade200,
)
```

৮. Icon Widget

Icon Widget Flutter-এর built-in Material icon গুলো দেখায়। Flutter-এ হাজারেরও বেশি icon আছে Icons class-এর মধ্যে।

```
// সাধারণ Icon
Icon(Icons.home)

// Styled Icon
Icon(
  Icons.favorite,
  color: Colors.red,
  size: 32,
  semanticLabel: 'পছন্দের তালিকা',
)

// কিছু জনপ্রিয় Icon
Icons.home      // বাড়ি
Icons.person    // ব্যক্তি
Icons.search    // খোঁজা
Icons.settings  // সেটিংস
Icons.delete    // মুছে ফেলা
Icons.edit      // সম্পাদনা
Icons.add       // যোগ করা
Icons.check     // চেক
Icons.arrow_back // পেছনে যাওয়া
Icons.menu      // মেনু (hamburger)
```

৯. TextField ও TextFormField

TextField ব্যবহারকারীর কাছ থেকে লেখা ইনপুট নেওয়ার জন্য ব্যবহার হয়। TextFormField হলো Form-এর মধ্যে ব্যবহারের জন্য বিশেষ version যেখানে validation সহজে করা যায়।

গুরুত্বপূর্ণ Properties

Property	Type	কাজ
controller	TextEditingController	ইনপুট পড়া ও নিয়ন্ত্রণ
decoration	InputDecoration	label, hint, prefix, suffix, border
keyboardType	TextInputType	কোন keyboard খুলবে
obscureText	bool	password field-এর জন্য
onChanged	Function(String)	লেখা পরিবর্তনে callback
onSubmitted	Function(String)	Enter চাপলে callback
maxLines	int	বহু লাইনের input box

Code উদাহরণ

```
final _nameController = TextEditingController();

TextField(
  controller: _nameController,
  keyboardType: TextInputType.text,
  decoration: InputDecoration(
    labelText: 'আপনার নাম',
    hintText: 'যেমন: Md Aslam',
    prefixIcon: Icon(Icons.person),
    border: OutlineInputBorder(borderRadius: BorderRadius.circular(8)),
    filled: true,
    fillColor: Colors.grey.shade100,
  ),
  onChanged: (value) => print('লেখা হচ্ছে: $value'),
)

// মান পড়া
String name = _nameController.text;

// মান মুছে ফেলা
_nameController.clear();

// Password Field
```

```
TextField(  
  obscureText: true,  
  decoration: InputDecoration(labelText: 'পাসওয়ার্ড', prefixIcon: Icon(Icons.lock)),  
)
```

১০. ListView Widget

ListView একটি scrollable তালিকা তৈরি করে। একাধিক item উল্লম্বভাবে বা অনুভূমিকভাবে দেখানোর জন্য ব্যবহার হয়।

ListView-এর ধরন

- `ListView()` — সরাসরি children list দেওয়া যায়। ছোট তালিকার জন্য।
- `ListView.builder()` — বড় বা dynamic তালিকার জন্য। Lazy loading করে।
- `ListView.separated()` — প্রতিটি item-এর মাঝে separator দেওয়া যায়।
- `ListView.custom()` — সম্পূর্ণ custom control।

Code উদাহরণ

```
// সাধারণ ListView
ListView(
  children: [
    ListTile(title: Text('আইটেম ১')),
    ListTile(title: Text('আইটেম ২')),
    ListTile(title: Text('আইটেম ৩')),
  ],
)

// ListView.builder - বড় তালিকার জন্য (recommended)
final List<String> students = ['Aslam', 'Rafi', 'Nadia', 'Hasan'];

ListView.builder(
  itemCount: students.length,
  itemBuilder: (context, index) {
    return ListTile(
      leading: CircleAvatar(child: Text('${index + 1}')),
      title: Text(students[index]),
      subtitle: Text('শিক্ষার্থী ID: 2024${index + 1}'),
      trailing: Icon(Icons.arrow_forward_ios),
      onTap: () => print('${students[index]} এ ক্লিক'),
    );
  },
)

// ListView.separated
ListView.separated(
  itemCount: 10,
  separatorBuilder: (ctx, i) => Divider(color: Colors.grey),
```

```
    itemBuilder: (ctx, i) => ListTile(title: Text('আইটেম ${i + 1}')),
  )

// Horizontal ListView
ListView.builder(
  scrollDirection: Axis.horizontal,
  itemCount: 5,
  itemBuilder: (ctx, i) => Container(
    width: 120, margin: EdgeInsets.all(8), color: Colors.blue.shade100,
    child: Center(child: Text('কার্ড $i')),
  ),
)
```

১১. GridView Widget

GridView items গুলোকে grid (জালের মতো) আকারে সাজায়। Photo gallery, product listing — এই ধরনের UI-তে ব্যবহার হয়।

```
// Fixed column count
GridView.builder(
  gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: 2,          // ২টি column
    crossAxisSpacing: 10,       // column-এর মাঝে ফাঁকা
    mainAxisSpacing: 10,        // row-এর মাঝে ফাঁকা
    childAspectRatio: 0.75,     // উচ্চতা:প্রস্থ অনুপাত
  ),
  itemCount: products.length,
  itemBuilder: (context, index) {
    return Card(
      child: Column(children: [
        Expanded(child: Image.network(products[index].imageUrl, fit: BoxFit.cover)),
        Text(products[index].name),
        Text('${products[index].price}'),
      ]),
    );
  },
)
```

১২. Card Widget

Card Widget একটি ছোট্ট কার্ড তৈরি করে যার চারদিকে ছায়া (shadow) থাকে। Material Design-এর card component।

```
Card(  
  elevation: 4, // ছায়ার গভীরতা  
  margin: EdgeInsets.all(8),  
  shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),  
  color: Colors.white,  
  child: Padding(  
    padding: EdgeInsets.all(16),  
    child: Column(  
      crossAxisAlignment: CrossAxisAlignment.start,  
      children: [  
        Text('শিক্ষার্থীর তথ্য', style: TextStyle(fontSize: 18, fontWeight:  
FontWeight.bold)),  
        SizedBox(height: 8),  
        Text('নাম: Md Aslam Howlader'),  
        Text('আইডি: 20245103012'),  
        Text('বিশ্ববিদ্যালয়: BUBT'),  
        SizedBox(height: 12),  
        ElevatedButton(onPressed: () {}, child: Text('বিস্তারিত দেখুন')),  
      ],  
    ),  
  ),  
)
```

১৩. AppBar Widget

AppBar স্ক্রিনের উপরে নেভিগেশন বার দেখায়। এটি Scaffold-এর appBar property-তে ব্যবহার হয়।

```
AppBar(  
  title: Text('আমার অ্যাপ'),  
  centerTitle: true,  
  backgroundColor: Colors.blue,  
  foregroundColor: Colors.white,  
  elevation: 4,  
  leading: IconButton(  
    icon: Icon(Icons.menu),  
    onPressed: () {},  
  ),  
  actions: [  
    IconButton(icon: Icon(Icons.search), onPressed: () {}),  
    IconButton(icon: Icon(Icons.more_vert), onPressed: () {}),  
  ],  
  bottom: TabBar(tabs: [  
    Tab(text: 'হোম'),  
    Tab(text: 'প্রোফাইল'),  
  ]),  
)
```

১৪. Navigator — স্ক্রিন পরিবর্তন

Navigator ব্যবহার করে এক স্ক্রিন থেকে আরেক স্ক্রিনে যাওয়া যায়। এটি stack-এর মতো কাজ করে — push করলে নতুন স্ক্রিন আসে, pop করলে আগেরটায় ফেরে।

```
// নতুন স্ক্রিনে যাওয়া
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => DetailScreen()),
);

// আগের স্ক্রিনে ফেরা
Navigator.pop(context);

// ডেটা পাঠানো
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => DetailScreen(name: 'Aslam', id: '20245103012'),
  ),
);

// সব স্ক্রিন বন্ধ করে নতুন একটিতে যাওয়া (logout-এর সময়)
Navigator.pushAndRemoveUntil(
  context,
  MaterialPageRoute(builder: (ctx) => LoginScreen()),
  (route) => false,
);
```

১৫. অন্যান্য গুরুত্বপূর্ণ Widget

SizedBox

নির্দিষ্ট আকারের ফাঁকা জায়গা তৈরি করতে বা Widget-এর আকার নির্ধারণ করতে ব্যবহার হয়।

```
SizedBox(height: 16)           // উল্লম্ব ফাঁকা
SizedBox(width: 8)             // অনুভূমিক ফাঁকা
SizedBox(width: 200, height: 50, child: ElevatedButton(...))
```

Padding

Widget-এর চারদিকে নির্দিষ্ট পরিমাণ ফাঁকা জায়গা যোগ করে।

```
Padding(
  padding: EdgeInsets.all(16),
  child: Text('আমার চারদিকে ফাঁকা আছে'),
)

// EdgeInsets সংক্ষেপ
EdgeInsets.all(16)           // চারদিকে সমান
EdgeInsets.symmetric(h: 16, v: 8) // horizontal ও vertical
EdgeInsets.only(left: 16, top: 8)  // নির্দিষ্ট দিক
EdgeInsets.fromLTRB(16, 8, 16, 8) // Left, Top, Right, Bottom
```

Expanded ও Flexible

Row বা Column-এর মধ্যে উপলব্ধ জায়গা সমানভাবে ভাগ করার জন্য Expanded ব্যবহার হয়।

```
Row(children: [
  Expanded(child: ElevatedButton(onPressed: () {}, child: Text('বাতিল'))),
  SizedBox(width: 10),
  Expanded(child: ElevatedButton(onPressed: () {}, child: Text('সাবমিট'))),
])
```

Wrap

Wrap Row-এর মতো কিন্তু জায়গা শেষ হলে নতুন লাইনে চলে যায়।

```
Wrap(
  spacing: 8,
  runSpacing: 4,
  children: ['Flutter', 'Dart', 'Firebase', 'Android'].map(
    (tag) => Chip(label: Text(tag))
  ).toList(),
)
```

SingleChildScrollView

যখন content স্ক্রিনের চেয়ে বড় হয়ে যায়, তখন scroll করার সুবিধা দেয়।

```
SingleChildScrollView(  
  child: Column(  
    children: [/* অনেক widget */],  
  ),  
)
```

GestureDetector

যেকোনো Widget-এ tap, swipe, long press সহ বিভিন্ন gesture detect করার জন্য।

```
GestureDetector(  
  onTap: () => print('ট্যাপ হয়েছে'),  
  onLongPress: () => print('দীর্ঘক্ষণ চাপা হয়েছে'),  
  onDoubleTap: () => print('দুইবার ট্যাপ'),  
  child: Container(width: 100, height: 100, color: Colors.blue),  
)
```

CircularProgressIndicator ও LinearProgressIndicator

Loading-এর সময় ব্যবহারকারীকে দেখানোর জন্য।

```
// বৃত্তাকার loading  
CircularProgressIndicator(color: Colors.blue)  
  
// রেখা আকারের loading  
LinearProgressIndicator(  
  value: 0.7, // 70% সম্পন্ন  
  backgroundColor: Colors.grey.shade200,  
  valueColor: AlwaysStoppedAnimation(Colors.green),  
)
```

Chip

ছোট তথ্য-ট্যাগ বা filter button হিসেবে ব্যবহার হয়।

```
Chip(  
  avatar: CircleAvatar(child: Text('A')),  
  label: Text('Flutter'),  
  backgroundColor: Colors.blue.shade100,  
  deleteIcon: Icon(Icons.close),  
  onDelete: () {},  
)
```

Switch, Checkbox & Radio

```
// Switch
Switch(value: _isDark, onChanged: (val) => setState(() => _isDark = val))

// Checkbox
Checkbox(value: _isChecked, onChanged: (val) => setState(() => _isChecked = val!))

// Radio
Radio<String>(
  value: 'male',
  groupValue: _gender,
  onChanged: (val) => setState(() => _gender = val!),
)
```

সারসংক্ষেপ — সব Widget একনজরে

নিচের ছকে সকল আলোচিত Widget-এর সংক্ষিপ্ত পরিচয় দেওয়া হলো:

#	Widget	প্রধান কাজ	কখন ব্যবহার করবেন
১	Text	লেখা দেখানো	যেকোনো লেখা, শিরোনাম, বার্তা
২	Container	বাক্স তৈরি করা	styling, padding, border, background
৩	Row	পাশাপাশি সাজানো	horizontal layout
৪	Column	উপর-নিচ সাজানো	vertical layout
৫	Stack	একের উপর আরেকটি রাখা	badge, overlay, layered UI
৬	Scaffold	পুরো স্ক্রিনের কাঠামো	প্রতিটি screen-এর root widget
৭	ElevatedButton	উঁচু বাটন	primary action
৮	TextButton	সমতল বাটন	secondary/link action
৯	IconButton	আইকন বাটন	toolbar, AppBar actions
১০	Image	ছবি দেখানো	photo, banner, avatar
১১	Icon	আইকন দেখানো	visual cue, button icon
১২	TextField	টেক্সট ইনপুট	form fields, search bar
১৩	ListView	স্ক্রল তালিকা	chat list, news feed, menu
১৪	GridView	গ্রিড তালিকা	gallery, product grid
১৫	Card	ছায়াযুক্ত বাক্স	profile card, product card
১৬	AppBar	উপরের নেভিগেশন বার	প্রতিটি screen-এ
১৭	SizedBox	ফাঁকা জায়গা	spacing between widgets
১৮	Padding	চারদিকে ফাঁকা	content padding
১৯	Expanded	বাকি জায়গা নেওয়া	Row/Column-এর মধ্যে
২০	GestureDetector	gesture ধরা	tap, swipe, long press

পরিশেষ (Conclusion)

Flutter-এর Basic Widget গুলো ভালোভাবে জানলে যেকোনো UI তৈরি করা সম্ভব। প্রতিটি Widget-এর নিজস্ব বৈশিষ্ট্য আছে এবং সেগুলো একে অপরের সাথে মিলিয়ে ব্যবহার করেই সুন্দর অ্যাপ তৈরি হয়।

মনে রাখবেন:

- Flutter-এ সবকিছুই Widget — UI থেকে শুরু করে layout পর্যন্ত।
- StatelessWidget ব্যবহার করুন যখন UI পরিবর্তন হবে না।
- StatefulWidget ব্যবহার করুন যখন setState() দিয়ে UI update করতে হবে।
- ListView.builder() সবসময় সাধারণ ListView-এর চেয়ে ভালো — এটি performance বাড়ায়।
- Container-এ color ও decoration একসাথে দেবেন না।
- Expanded শুধুমাত্র Row বা Column-এর সরাসরি child হিসেবে ব্যবহার করুন।

শুভকামনা, Md Aslam! Flutter-এ আপনার যাত্রা সফল হোক!